11/28/2023

MY PLANT CARE ASSISTANT

A Python application using Stremlit

SOFIA CALDERON

AGENDA ★

- Introduction
- Application Overview
- Code Demo
- Strengths/Weaknesses
- Next Steps

Streamlit

python™

INTRODUCTION

Back to Agenda Page

🌱 My Plant Care Assistant 📝

Purpose: Assist users in plant care by managing tasks related to the care of their plants, such as watering, fertilizing, or picking

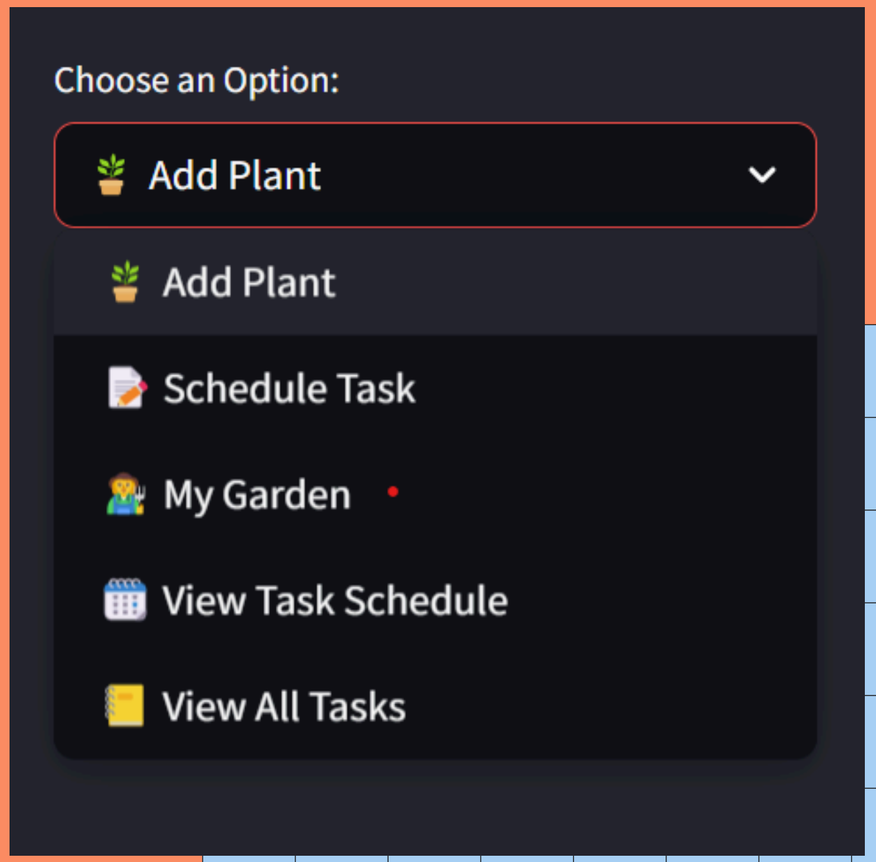
Key Features:

1. Add Plant
2. Schedule Task
3. My Garden
4. View Task Schedule
5. View All Tasks

APPLICATION OVERVIEW

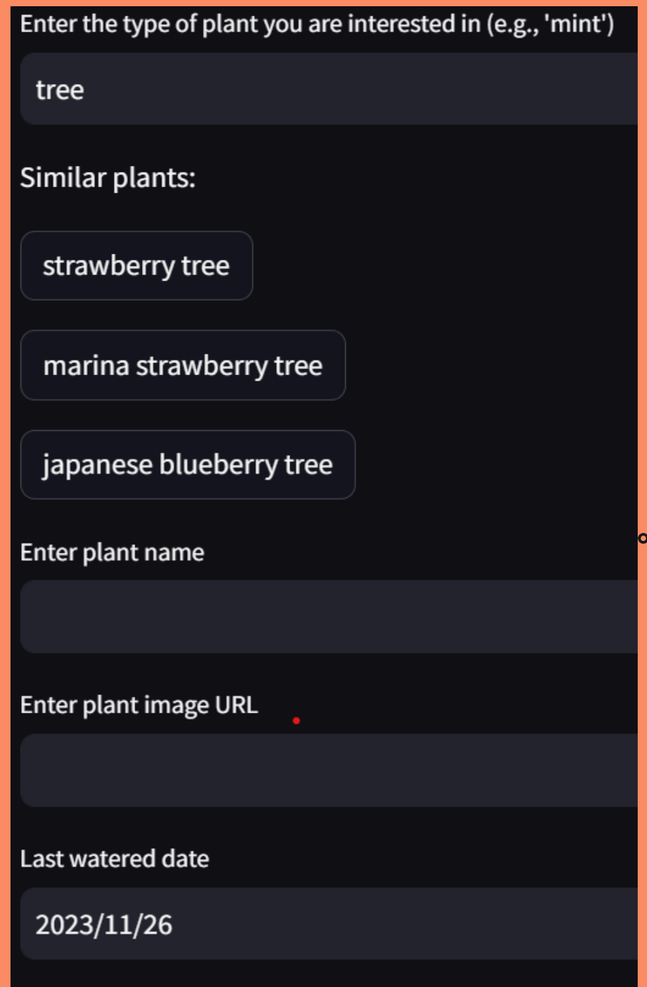
Back to Agenda Page

1. **Add Plant**: Users can add new plants using a plant database for suggestions.
2. **Schedule Task:** Users can schedule watering, fertilizing, and other tasks for their plants.
3. **My Garden:** Users can view their plant collection with images and watering recommendations.
4. **View Task Schedule:** Overview of the task scheduling feature.
5. **View All Tasks:** Users can see a comprehensive list of all scheduled tasks.



ADD PLANT

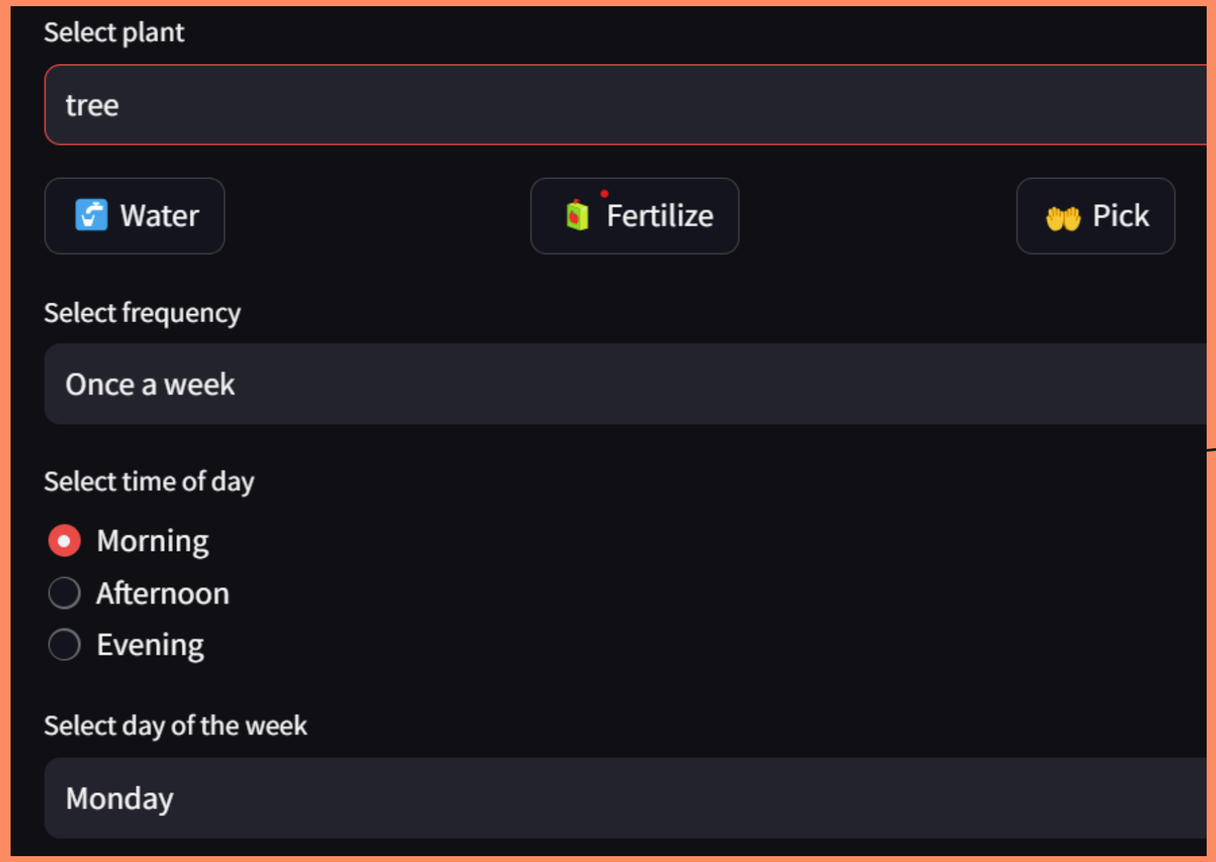
Back to Agenda Page



- Users can input a plant name and image URL.
- Plant database used for suggestions.

SCHEDULE TASK

Back to Agenda Page



- Users can schedule tasks for specific plants.
- Selection of task type (watering, fertilizing, picking).
- Frequency options: Once a week, multiple times a week, once a month.
- Time of day selection (Morning, Afternoon, Evening

MY GARDEN

Back to Agenda Page

My Garden

	0
Frequent	Recommended to water 2-3 times a week
Average	Recommended to water at least once a week
Minimum	Recommended to water once a month
None	Can live with minimal or no water

tree

tree

- Showcase a table with plant images and names.
- Watering recommendations for each plant: Frequent, Average, Minimum, None.
- Explanation of the toggleable plant selection for detailed information.

MY GARDEN

Back to Agenda Page

tree

tree

Details for tree:

Last Watered: 2023-11-26

Plant in Database: strawberry tree

Suggested Watering: frequent

- Showcase a table with plant images and names.
- Watering recommendations for each plant: Frequent, Average, Minimum, None.
- Explanation of the toggleable plant selection for detailed information.

VIEW TASK SCHEDULE

Back to Agenda Page

Tasks for Today

	Task	Plant	Date	Time of Day
0	Fertilize	tree	Sunday	Afternoon

Weekly Tasks

	Task	Plant	Date	Time of Day
0	Water	tree	Monday	Morning

- Display tasks for the day, week, and month.
- Differentiate between daily, weekly, and monthly tasks.
- Overview of the optimized task schedule.

VIEW ALL TASK SCHEDULE

Back to Agenda Page

Task	Plant	Date	Time of Day	
🚰 Water	tree	Monday	Morning	Delete
🧃 Fertilize	tree	Sunday	Afternoon	Delete
🙌 Pick	tree	5 of each month	Morning	Delete

How do you want to sort your tasks:

Day of Week

Select which day of the week

Sunday

Sorted Tasks By: Day of Week

	Task	Plant	Date	Time of Day
0	🧃 Fertilize	tree	Sunday	Afternoon

- Users can view and delete scheduled tasks.
- Sorting options: Day of the week, Monthly tasks, Time of day.
- Displays a table with sorted tasks for clarity.

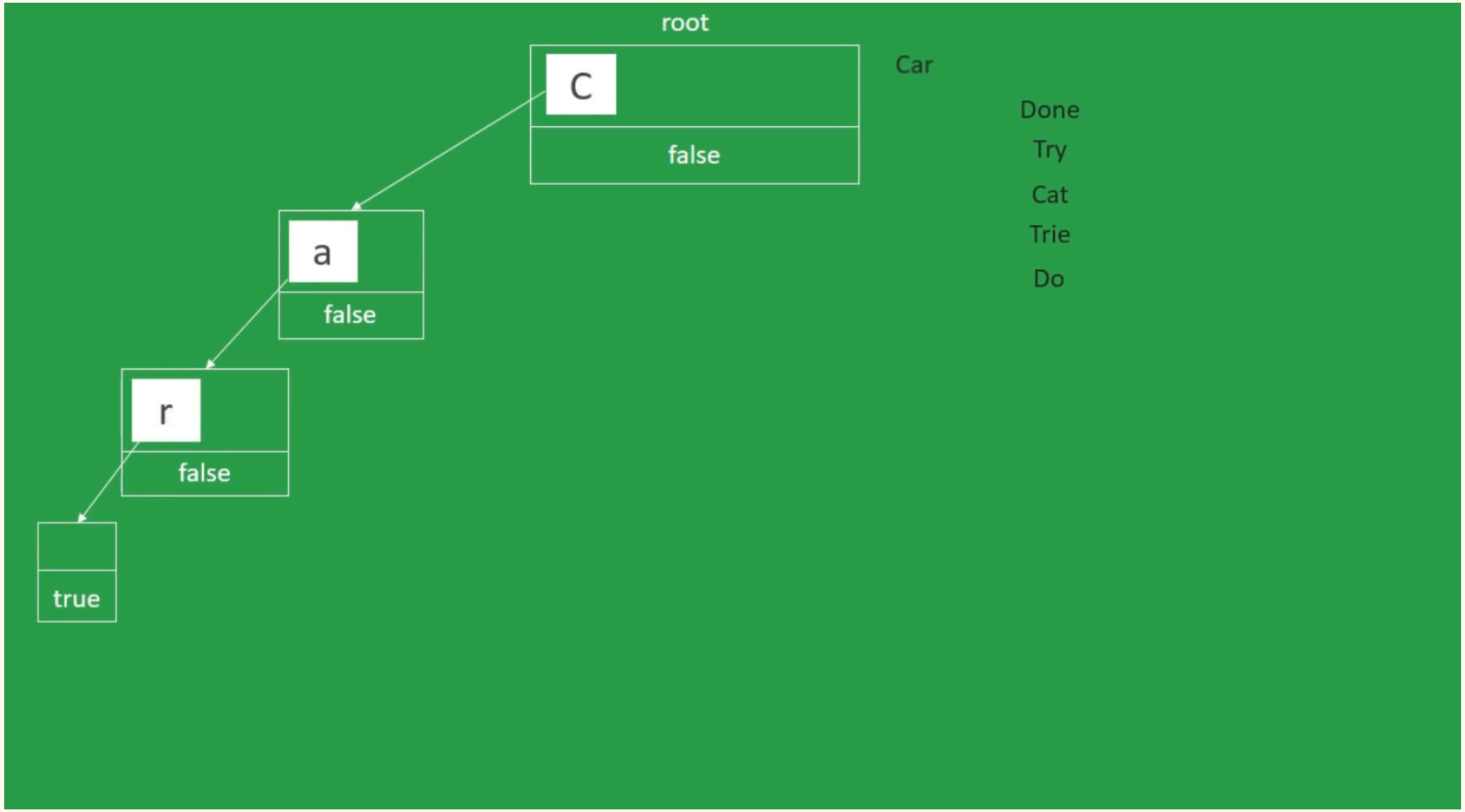
DATA STRUCTURES & ALGORITHM

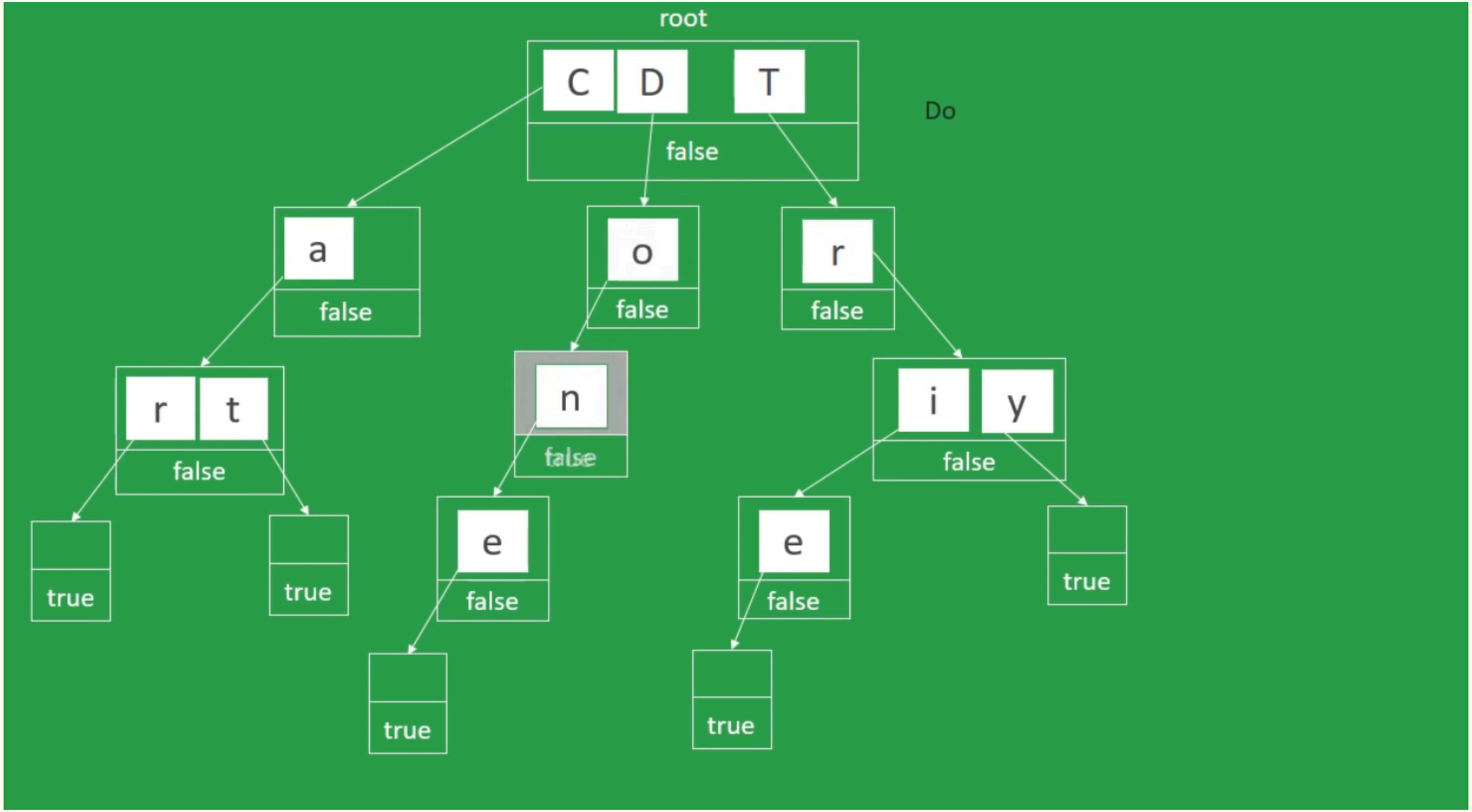
Back to Agenda Page

```python
You, 2 days ago | 1 author (You)
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

You, 2 days ago | 1 author (You)
class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def insert_with_reversed_words(self, name):
        reversed_name = ' '.join(name.split()[::-1])  # Reverse the order of words
        node = self.root
        for char in reversed_name:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, prefix):
        node = self.root
        for char in prefix:
            if char not in node.children:
                return []
            node = node.children[char]
        return self._get_words_from_node(node, prefix)

    def _get_words_from_node(self, node, prefix):    sofia calderon [2 days ago] • initial
        words = []
        if node.is_end_of_word:
            words.append(prefix)
        for char, next_node in node.children.items():
            words.extend(self._get_words_from_node(next_node, prefix + char))
        return words

def build_plant_trie(plant_names):
    trie = Trie()
    for name in plant_names:
        trie.insert_with_reversed_words(name.lower())
    return trie
```

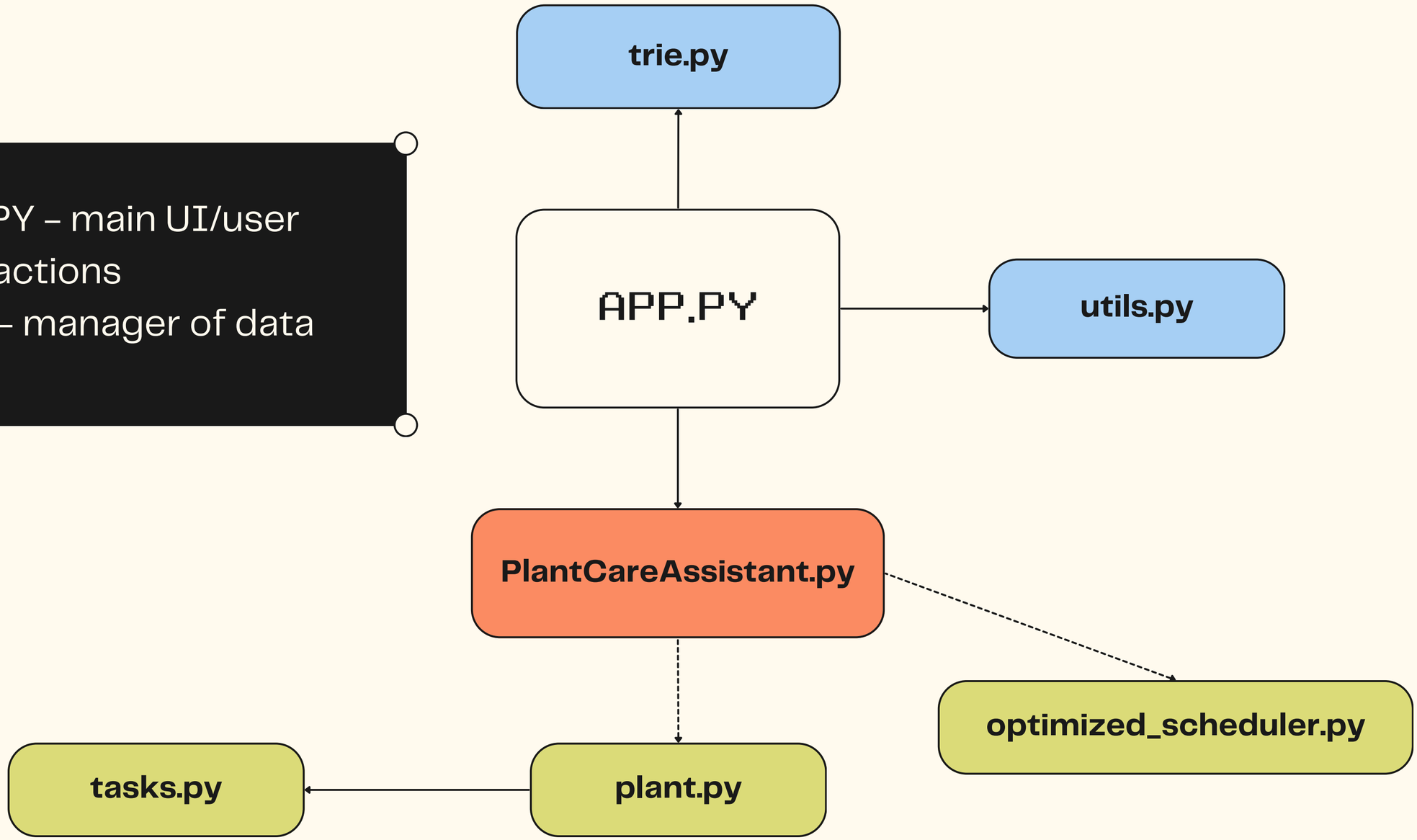
- Trie (to find prefix – in this case second word or only word)
- Lists
- Dictionaries
- Queue

- Merge sort (when sorting/filtering tasks)


root
C
false
a
false
r
false
true
Car
Done
Try
Cat
Trie
Do


root
C
D
T
false
Do
a
false
o
false
r
false
r
t
false
n
false
i
y
false
true
true
e
false
e
false
true
true
true

- APP.PY – main UI/user interactions
- PCA – manager of data



Back to Agenda Page

WEAKNESSES

- Limited Plant Database
- No Task Editing
- User Feedback

MY PLANT CARE ASSISTANT

SAVE

STRENGTHS

- Adaptive Structure
- Easy Modification
- User Interface

NEXT STEPS

Back to Agenda Page

Implement Adaptive Scheduler: Creates a schedule that takes into account watering rates stored in the database.

Implement New User Features: Ability to edit tasks, better user feedback,